Gesture Sentry: Gesture-Controlled Sentry Turret with Drive Base

Final report · concept to realisation · due end of October · 15–20 pages excluding front matter and appendices

Scope tier	Core tier: glove-to-turret BLE control with rule-based gestures, fail-safe wheel task and web control. Not delivered: TinyML and AUTO tracking *[confirm tier wording with your guide]*
Submission date	27 / 10 / 2026
Repository link	Local project folder `ESA_Friend_Project` (not a git repository). *[add GitHub URL if pushed]*
Demo video link	*[add link]*

TEAM

Member	Reg. no.	Contribution (module owned)	Sections written
Sankepalli Kharanshu	25BCE2093	Complete hardware build and calibration of the main (turret) ESP32	§3.2 (pin map), §3.4, §9 (inputs)
Tom Biju	25BCE0973	Glove code calibration: sensor orientation learning and zero-point calibration	§4.1 (inputs), §7 T1–T3 (inputs)
Ahmed Sahil	25BCE0252	Documentation and report creation	All sections: assembly, §1, §2, §8, §10, §11, abstract
Harish Saravanan	25BCE0987	Wheel-drive testing	§7 T4, T4b, T5 (inputs)

DECLARATION

We declare that this report and the work it describes are our own, and that every external source, library and tool is acknowledged in the references.

Member	Date
Sankepalli Kharanshu	27 / 10 / 2026
Tom Biju	27 / 10 / 2026
Ahmed Sahil	27 / 10 / 2026
Harish Saravanan	27 / 10 / 2026

Abstract

Hands-busy remote operation needs a controller that is worn, not held. We built a two-ESP32 system: a glove carrying an MPU6500 inertial sensor, and a turret with a pan-tilt servo mount, laser, OLED, RGB indicator, buzzer, ultrasonic and PIR sensors, and an L298N four-wheel base. The glove sends roll, pitch and gesture events over BLE every 40 ms. The turret turns them into pan/tilt or drive commands, and can instead be driven from a phone over its own WiFi access point. Gesture recognition is a rule-based detector on filtered IMU data; it replaced the TinyML model in the proposal. On synthetic traces (n = 300 per gesture) it recognised double flick, punch and shake 100% of the time and sweeps 94.7–95.3%, against a 90% target, with no false actions in a simulated hour. The wheel arming and watchdog logic passes host tests and the driver pins were verified on hardware. The main limitations: sweeps are not detected reliably on the real hand, and latency, task timing and battery current were not measured.

Contents

Abstract . . . 2

Contents . . . 2

1 Problem and requirements . . . 4

1.1 Problem statement . . . 4

1.2 Requirements . . . 4

1.3 Constraints . . . 4

2 Alternatives considered . . . 5

2.1 Microcontroller . . . 5

2.2 Glove-to-turret link . . . 5

2.3 Gesture recognition method . . . 5

2.4 Turret software architecture . . . 6

3 Architecture and hardware . . . 6

3.1 System architecture . . . 6

3.2 Schematic and pin map . . . 7

3.3 Power budget . . . 8

3.4 Build . . . 9

4 Design models . . . 9

4.1 CDFG and critical path . . . 10

4.2 System FSM . . . 11

4.3 Task schedule . . . 12

4.4 Changes since Review 2 . . . 13

5 Firmware structure . . . 13

5.1 Layers and modules . . . 14

5.2 Implementation of the models . . . 14

5.3 Model-to-code traceability . . . 15

6 Optimisation . . . 15

7 Test plan and measured results . . . 16

7.1 Timing measurements . . . 17

7.2 Discussion . . . 17
8 Design-decision log . . . 17
9 Bill of materials and cost . . . 18
10 Individual contribution . . . 19
11 Limitations and future work . . . 19
References . . . 20
Appendices . . . 21
A Source code . . . 21
B Raw data . . . 21
C Review 2 models (changed substantially) . . . 22
Submission checklist . . . 23

1 Problem and requirements

1.1 Problem statement

Small robots and turrets are normally driven with a joystick, a button panel or a phone. All three need a hand and the operator's eyes on the controller, and none of them uses the motion of the hand itself. We want a worn controller: tilting the wrist steers the machine, and a few distinct wrist motions switch mode or fire a one-second laser pulse. Because the operator cannot look at the controller, the machine has to be safe on its own: nothing may move until the hand is at neutral, a lost link must stop every output, and a mis-detected gesture must never leave the laser on. The intended users are demonstrators and hobbyists; per the Review 1 condition we present it as a tracking and pointing platform: the laser is a low-power visible pointer module, nothing is projectile, and the "sentry" in the approved title is a project name only.

1.2 Requirements

ID	Requirement	Measurable target	Priority (must / should)	Verified in
R-F1	Glove roll and pitch command turret pan and tilt	Pan 30–150°, tilt 45–135°. At rest after calibration: mean within ±3° of 90°, sd ≤ 2°	must	§7 T1
R-F2	Glove recognises five action gestures: double flick (laser 1 s), shake (drive/turret toggle), punch (lock flash), sweep left, sweep right	≥ 90% exact-match per gesture; 0 false actions in a simulated hour of normal motion	must	§7 T2, T3
R-F3	Glove drives four wheels with an arming rule	Wheels off at boot and until the first neutral sample; PWM 90–200 of 255; 150 ms dead time on reversal	must	§7 T4
R-F4	Phone or laptop can drive the car over the turret's own WiFi	Page loads at 192.168.4.1; motors stop ≤ 400 ms after the last command	should	§7 T5
R-F5	Ultrasonic and PIR raise a visible/audible alert	Alert below 30 cm; range repeatable to ±1 cm at a fixed target; PIR after 60 s warm-up	should	§7 T6
R-N1	End-to-end latency, glove motion to actuator	≤ 300 ms	must	§7 T7
R-N2	Fail-safe behaviour	Laser ≤ 1 s per trigger and off within 300 ms of a stale link; wheels stop ≤ 250 ms after link loss or main-loop stall	must	§7 T8
R-N3	Cost	≤ ₹5,000 total	must	§9, §7 T9
R-N4	Untethered operation	≥ 10 min on batteries, both boards	should	§7 T10

F = functional (what it does). N = non-functional (latency, accuracy, power, cost, size).

1.3 Constraints

Budget: ₹5,000 for the whole team (spent: ₹3,462 on three receipts dated 28-09-2026). **Hardware:** only what was bought: two ESP32 dev boards, MG995 and SG90 servos, MPU6050-type breakout (the chip on it is an MPU6500), HC-SR04, HC-SR501, laser module, buzzer, 0.96" OLED, two RGB modules, four 18650 cells with a four-cell holder and a two-cell charger, breadboards, perfboard, jumpers and a 4WD acrylic chassis; an L298N driver was added for the wheels. **Power:** no regulator for the battery pack and no battery for the glove were bought, so both boards run on USB. **Time:** about four weeks from the proposal; firmware bring-up happened on 26–27 October. **Team:** four members, one module each. **Toolchain:** Arduino-ESP32 3.3.12 on Windows; the turret sketch only fits with the Huge APP partition scheme.

2 Alternatives considered

Scores are our judgement from datasheets and library documentation, 1 = poor, 5 = good; they are not measurements. Weights add to 100 and follow the requirement named in each row.

2.1 Microcontroller

Criterion (requirement ID)	Weight	A: ESP32-WROOM-32	B: Arduino Nano + HC-05	C: Raspberry Pi Pico W
Wireless built in (R-F1, R-N1)	25	5	2	4
GPIO and PWM count: 17+ pins, 2 servos, 2 wheel PWM (R-F3)	20	5	3	4
Cost (R-N3)	20	4	3	4
Toolchain and our experience (schedule)	15	5	4	3
CPU/FPU headroom for IMU filters (R-F2)	10	5	1	3
Power draw (R-N4)	10	2	3	3
Weighted total (1–5)	**100**	**4.50**	**2.70**	**3.65**

Chosen: A, ESP32-WROOM-32 (two boards, ~₹450 each). **Because:** it is the only option that has BLE, WiFi (for the access point), 17+ usable pins and a hardware FPU on one ₹450 board; B has no FPU and needs an external Bluetooth module and C was unfamiliar. Cost on power: ESP32 is the highest draw (§3.3).

2.2 Glove-to-turret link

Criterion (requirement ID)	Weight	A: BLE GATT read	B: ESP-NOW	C: WiFi UDP
Latency (R-N1)	30	3	5	4
Glove power (R-N4)	20	4	3	2
Pairing effort and coexistence with the web AP (R-F4)	15	3	4	2
Range (R-F1)	10	4	4	4
Library support in Arduino-ESP32 (schedule)	10	4	4	5
Fit with the approved proposal	15	5	2	2
Weighted total (1–5)	**100**	**3.70**	**3.80**	**3.10**

Chosen: A, BLE (glove advertises one characteristic, turret reads it every 50 ms). **Because:** it was the proposal's link, it works alongside the WiFi access point on the same radio, and it was verified end to end on 26 October (glove `BLE=CONNECTED`, turret `link=1`). The matrix puts ESP-NOW marginally ahead because of latency, so we record this honestly: switching is the first latency experiment in §11, not something we did before the deadline.

2.3 Gesture recognition method

Criterion (requirement ID)	Weight	A: TinyML 1D-CNN (Edge Impulse)	B: rule-based on filtered IMU	C: template matching (DTW)
Time to a working demo (schedule)	25	2	5	3
Training data needed (schedule)	15	1	5	3
Accuracy ceiling (R-F2)	20	5	3	4
Explainable, tunable on the bench (R-F2)	15	2	5	3
Flash, RAM and execution time on the glove ESP32	15	3	5	3
Course relevance (AI component)	10	5	2	3
Weighted total (1–5)	**100**	**2.90**	**4.30**	**3.20**

Chosen: B, a rule-based detector (`glove/Gestures.h`). **Because:** no labelled dataset was ever collected, so A could not be trained, and thresholds can be reasoned about and tuned from serial output in minutes. This departs

from the proposal; the cost is the AI component and a lower accuracy ceiling, and we say so in §11.

2.4 Turret software architecture

Criterion (requirement ID)	Weight	A: single super-loop	B: super-loop + one RTOS task	C: one RTOS task per subsystem
Safety isolation of wheel outputs (R-N2)	30	1	4	5
Simplicity and time to build (schedule)	20	5	4	2
Latency (R-N1)	20	2	3	5
Debuggability over serial (schedule)	15	5	4	2
RAM and flash (R-N3 headroom)	15	5	4	3
Weighted total (1–5)	**100**	**3.20**	**3.80**	**3.65**

Chosen: B. **Because:** the only output that can hurt something is the motor driver, so it alone gets its own FreeRTOS task (priority 2, 10 ms) with an independent 250 ms watchdog. Everything else stays in a simple millis-gated loop that is easy to read and debug. C scored higher on safety and latency but costs more than we had.

3 Architecture and hardware

3.1 System architecture

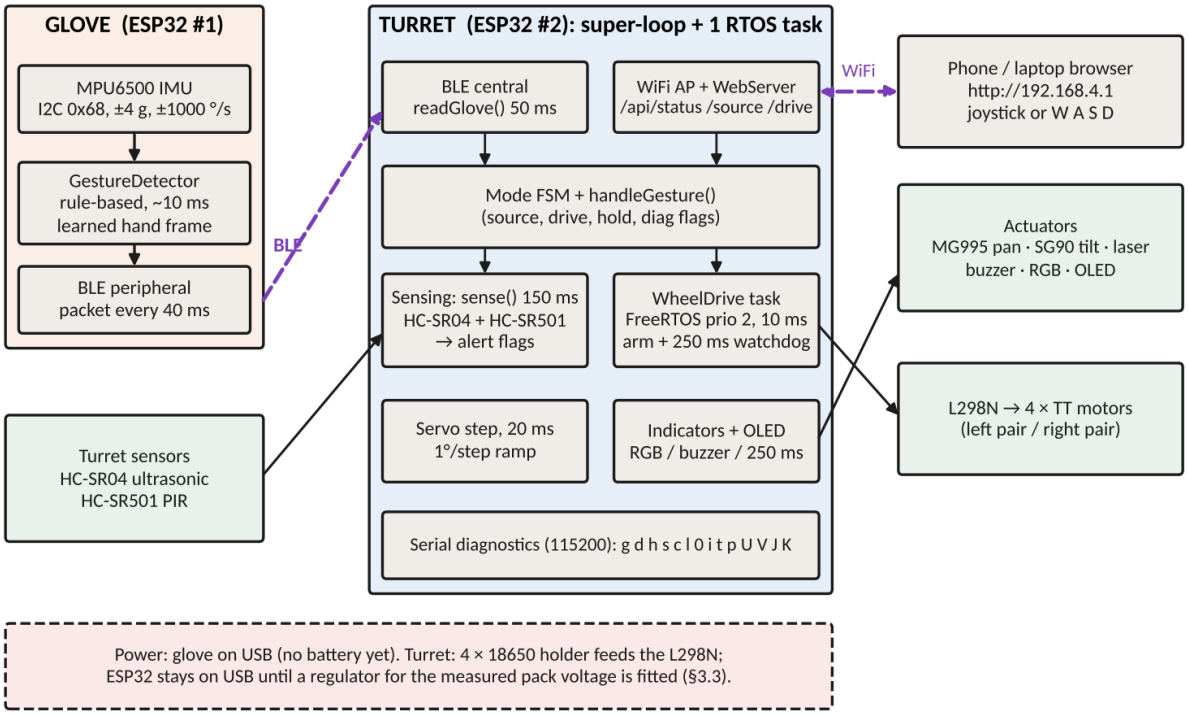


Figure 1 — System block diagram, final version. Dashed purple: wireless links.

The glove samples the IMU every ~10 ms, runs the gesture detector, and refreshes one BLE characteristic every 40 ms with `seq`, `pan`, `tilt`, `evSeq`, `evCode`. The turret polls that characteristic every 50 ms, rejects anything out of range or malformed, and acts on a gesture only if its event counter changed after the turret connected. A second command source, the web page, is mutually exclusive with the glove: selecting it stops both paths so that no stale command carries over.

`BLE packet specification`

Field	Value
Link	BLE GATT: glove is the peripheral and advertises one service with one readable characteristic; the turret is the central, connects to the advertised service UUID and reads the characteristic every 50 ms (the glove refreshes it every 40 ms)
Payload	ASCII text, `seq,pan,tilt,evSeq,evCode`, for example `1042,91,89,17,4`. The three-field form `seq,pan,tilt` is also accepted (older glove firmware)
seq	Unsigned counter, +1 per packet. A packet with the same seq as the last one is a repeat and is ignored. It is also the freshness signal: no new seq for 300 ms means the link is stale
pan, tilt	Integers in degrees. Valid pan 30–150, tilt 45–135, otherwise the whole packet is dropped
evSeq, evCode	Gesture event counter and code. Code 0 none, 1 double flick, 2 shake, 3 still (report only), 4 punch, 5 sweep left, 6 sweep right; valid 0–6. A gesture acts only if evSeq changed after the turret connected, so a gesture made before connecting is never replayed
Malformed packet	Any packet that does not parse completely, or has a field out of range, is dropped; the previous good command is kept until the 300 ms staleness rule fires

3.2 Schematic and pin map

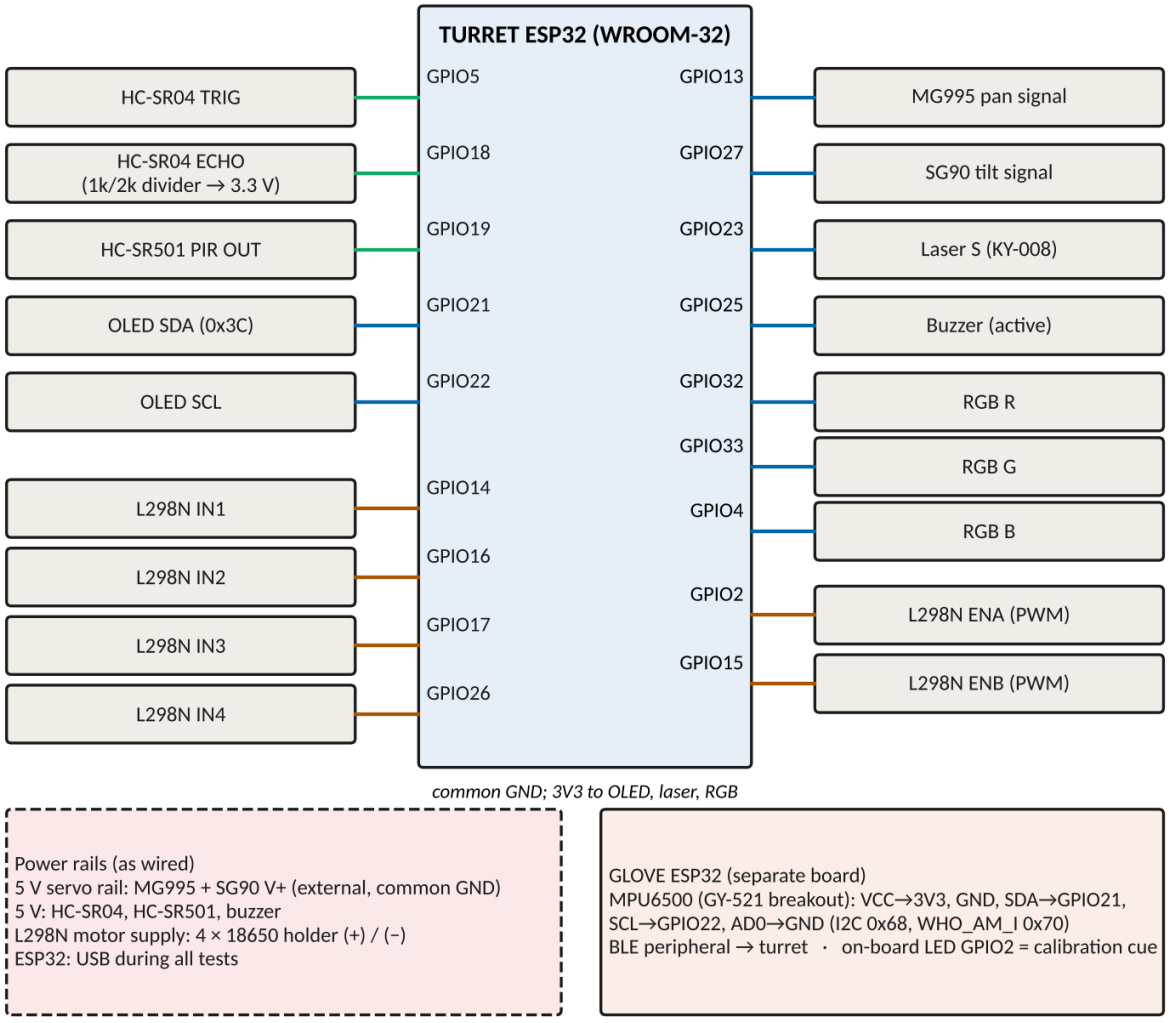


Figure 2 — Circuit connections, as wired (turret ESP32 centre; glove board at bottom right).

MCU pin	Function	Connected to	Notes (pull-up, level shift, interrupt)
GPIO13	PWM 50 Hz (ESP32Servo)	MG995 pan signal	Servo V+ from external 5 V rail, common GND
GPIO27	PWM 50 Hz	SG90 tilt signal	GPIO12 avoided (boot strap pin)

MCU pin	Function	Connected to	Notes (pull-up, level shift, interrupt)
GPIO5 / GPIO18	Digital out / in	HC-SR04 TRIG / ECHO	ECHO through 1 kΩ + 2 kΩ divider (5 V → 3.3 V); `pulseIn` timeout 25 ms
GPIO19	Digital in	HC-SR501 PIR OUT	3.3 V logic; ignored for the first 60 s
GPIO21 / GPIO22	I2C SDA / SCL	OLED SSD1306, 0x3C	`Wire.setTimeOut(25)`; library switches to 400 kHz during `display()`
GPIO23	Digital out	Laser module S	Low at boot; forced low whenever the link is stale
GPIO25	Digital out	Active buzzer	Transistor driver recommended for 5 V buzzer
GPIO32 / 33 / 4	Digital out	RGB module R / G / B	Common-cathode assumed (`RGB_COMMON_ANODE=false`)
GPIO14 / 16 / 17 / 26	Digital out	L298N IN1 / IN2 / IN3 / IN4	Direction; GPIO16/17 unavailable on PSRAM boards
GPIO2 / GPIO15	LEDC 5 kHz, 8-bit	L298N ENA / ENB	Boot-strap pins: jumpers removed; ENA on GPIO2 can block flashing ("wrong boot mode 0xb")
Glove GPIO21 / 22	I2C SDA / SCL	MPU6500 (GY-521), 0x68	AD0 to GND; WHO_AM_I = 0x70; on-board LED GPIO2 used as calibration cue

3.3 Power budget

Component	Supply voltage (V)	Active current	Sleep / idle current	Duty cycle	Average current
Turret ESP32 (WiFi AP + BLE central)	5 (USB)	180 mA	180 mA	100%	180.0 mA
MG995 pan servo	5	500 mA	10 mA	5%	34.5 mA
SG90 tilt servo	5	250 mA	6 mA	5%	18.2 mA
HC-SR04	5	15 mA	15 mA	100%	15.0 mA
HC-SR501 PIR	5	0.065 mA	0.065 mA	100%	0.1 mA
OLED SSD1306 (text)	3.3	15 mA	15 mA	100%	15.0 mA
RGB module (one colour)	3.3	8 mA	0	100%	8.0 mA
Laser module	3.3	30 mA	0	1%	0.3 mA
Active buzzer	5	30 mA	0	5%	1.5 mA
L298N logic supply	5	36 mA	36 mA	100%	36.0 mA
4 TT motors via L298N (driving)	pack	1000 mA	0	25%	250.0 mA
Turret total (mixed rails, indicative)					**559 mA**
Glove ESP32 (BLE peripheral)	5 (USB)	100 mA	100 mA	100%	100.0 mA
MPU6500	3.3	3.2 mA	3.2 mA	100%	3.2 mA
Glove total					**103 mA**

*Every current is a datasheet-typical or estimated value, **not measured**: no ammeter log or pack-voltage measurement was made. Duty cycles are our estimates of a demo session (servos moving 5% of the time, motors driving 25%, laser firing 1%). Rows are on different rails, so the totals only show that motor current dominates (≈250 of ≈559 mA) and that the ESP32 radios are the largest steady load. The four-cell holder's wiring (series or parallel) and the pack voltage were never measured, which is why no regulator was fitted; the ESP32 boards stay on USB (§11).*

3.4 Build

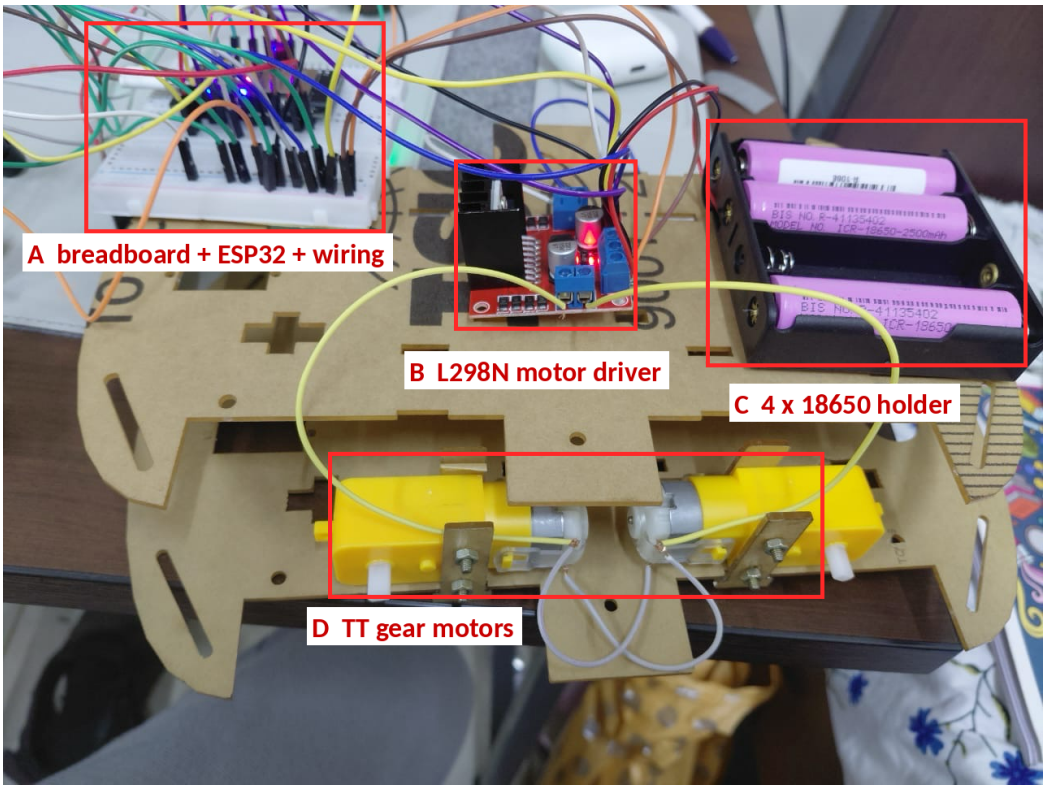


Figure 3 — Assembled hardware: (A) breadboard with the turret ESP32 and wiring, (B) L298N motor driver, (C) four-cell 18650 holder, (D) TT gear motors on the acrylic chassis.

The photograph shows the drive base. The pan-tilt bracket with the MG995 and SG90, the ultrasonic, PIR and laser mounting, and the glove strap were not yet mounted at the time of writing; those peripherals were tested on the breadboard only. The kit's bracket is sized for two MG995/MG996R servos and will need an adapter plate for the SG90 tilt servo.

4 Design models

4.1 CDFG and critical path

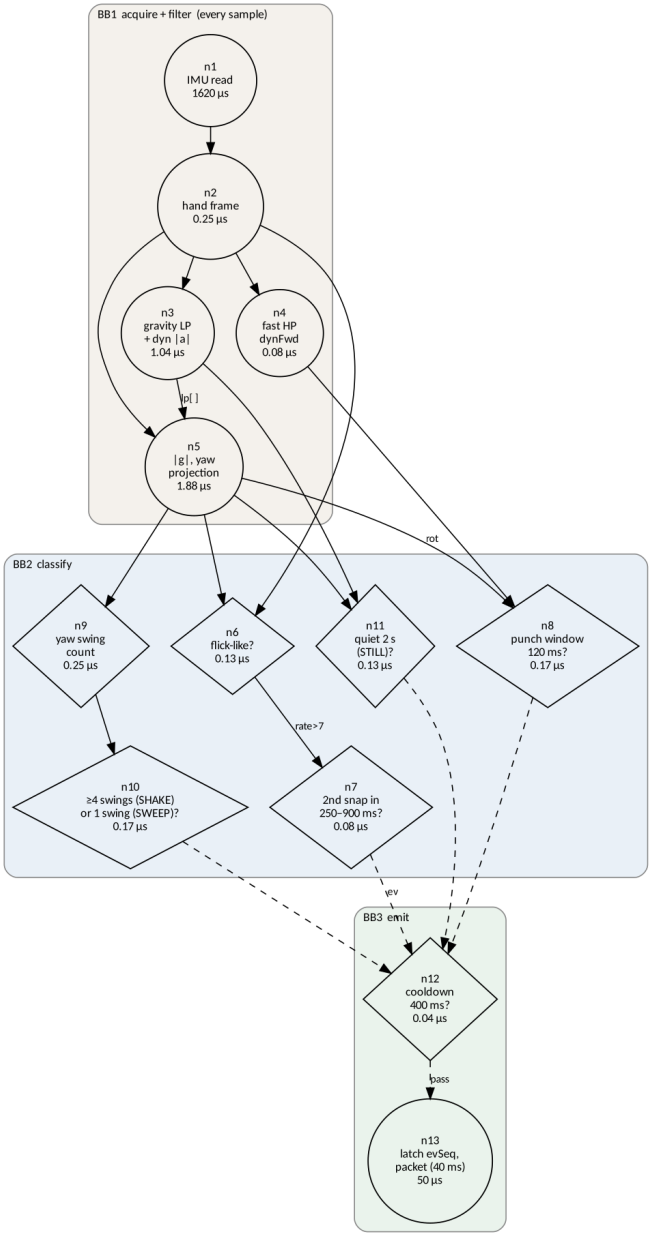


Figure 4 — CDFG of the critical function, final: one pass of the glove's 10 ms loop (IMU read plus `GestureDetector::update()`*). Circles = operations, diamonds = decisions, solid = data, dashed = control; costs are estimates.*

The critical function changed from the Review 2 plan (classifying a 1.5 s window with a neural network, estimated 24 ms) to this rule-based pass. Its critical path is n1 → n2 → n5 → n9 → n10 → n12 → n13: 1620 + 0.25 + 1.88 + 0.25 + 0.17 + 0.04 + 50 = **1673 µs** against the 10 ms sample period, so slack is 8.3 ms (n13 runs on one sample in four). The I2C burst read (n1, 14 bytes at 100 kHz) is 97% of it. Node costs are estimates from instruction counts at 240 MHz and bus arithmetic; they are not measured (§7.1).

4.2 System FSM

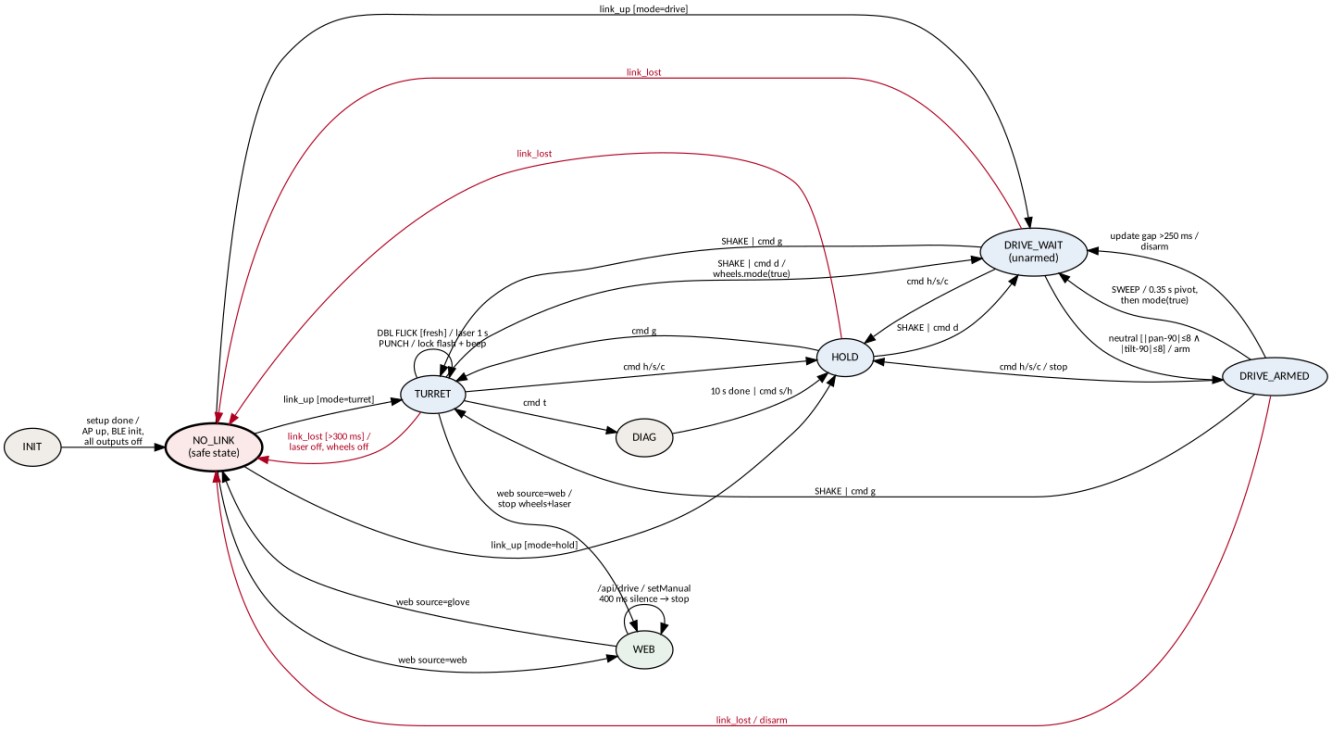


Figure 5 — System FSM, final. Red edges lead to NO_LINK, the safe state.

The turret is a Mealy machine: the same gesture does different things in different states (a double flick fires the laser only in TURRET). The state is not an enum in the code but the flag tuple `source`, `driveMode`, `holdPosition`, `diagnostic` plus the derived `fresh()`; Figure 5 names the combinations. In the safe state NO_LINK the laser pin is low, both wheel enables are at duty 0 and the direction pins low, the buzzer is off and the servos hold. It is reached from TURRET, HOLD, DRIVE_WAIT and DRIVE_ARMED when a packet is more than 300 ms old or the BLE client disconnects. In WEB the equivalent is the 400 ms command expiry inside the wheel task.

4.3 Task schedule

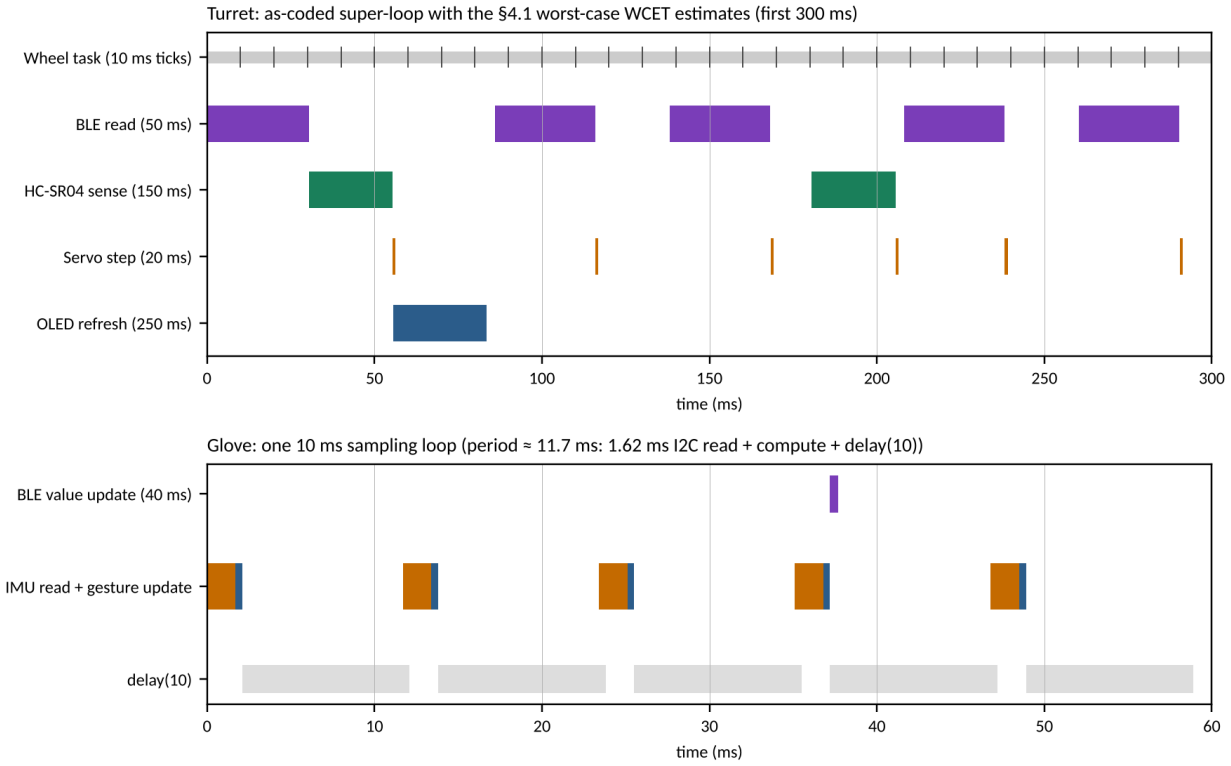


Figure 6 — Schedule timeline, final: turret super-loop (top) and glove sampling loop (bottom), built from the §4.1 WCET estimates, not from a capture.

Turret job	Period	WCET (est.)	Simulated start-to-start interval: mean / max	Comment
BLE read	50 ms	30 ms	59 / 85 ms	GATT read waits about one connection interval
HC-SR04 sense	150 ms	25 ms	166 / 184 ms	`pulseIn` blocks up to its 25 ms timeout
Servo step	20 ms	0.05 ms	55 / 100 ms	Runs late: the loop is blocked by the three jobs above
OLED refresh	250 ms	28 ms	276 / 298 ms	1 KB frame at 400 kHz
Wheel task (RTOS)	10 ms	0.1 ms	not affected	Own task, priority 2, preempts the loop

Utilisation of the loop jobs is U = 30/50 + 25/150 + 28/250 + 0.05/20 = 0.89. Because the loop is non-preemptive, the 20 ms servo step is delayed by up to 100 ms in this model, so the pan and tilt ramp at about 18°/s instead of the nominal 50°/s. This is a model result, not a measurement; the safety deadlines are unaffected because they sit in the wheel task and in the 300 ms link-freshness check.

4.4 Changes since Review 2

Model element	Review 2 version	As built	Why it changed
Gesture method (CDFG)	TinyML classifier on a 1.5 s window, 4 baseline gestures (+4 optional stretch), ≈24 ms (est.)	Rule-based detector, 6 events, ≈1.7 ms per sample (est.)	No labelled dataset was collected; rules are tunable from serial output (§2.3)
CDFG n8 punch	n/a	Judged 120 ms after onset; a rotation above 6 rad/s vetoes it	Swats and flicks were misread as punches on the real hand
CDFG n5 yaw	n/a	Yaw taken about the gravity axis, not the sensor z axis	A sweep with the hand edge-on gave no z rotation
System FSM: modes	IDLE, SCANNING, TRACKING, MANUAL, LOCKED, FAULT	NO_LINK, TURRET, HOLD, DRIVE_WAIT, DRIVE_ARMED, WEB, DIAG	AUTO tracking was not implemented (sensors only raise alerts); the 4WD base and web control were added
FSM: safe state	FAULT on stale BLE or watchdog	NO_LINK on stale BLE; 250 ms wheel watchdog; 400 ms web expiry	Safety moved into the wheel task so it works even if the main loop stalls
FSM: safe-state action	Servos centre, laser off, OLED says link lost (Review 1 condition)	Laser off, wheels stop, OLED shows `GLOVE LINK LOST`; servos **hold** the last position	Not implemented: `turret.ino` L324 comment "hold the last position on loss". Centring on loss is a few-line change
FSM: arming	n/a	Wheels arm on the first neutral sample (formerly a 500 ms hold)	Removes the wait; trade-off: the car may move if the glove reconnects near flat
FSM: STILL gesture	n/a	Reported only, no action (formerly stopped the car)	It clashed with the level hold needed to arm the wheels
Schedule	Rate-monotonic: 7 preemptive tasks, U = 0.61	Super-loop plus one RTOS task, loop jobs U ≈ 0.89	Simpler to debug; costs servo smoothness (§4.3)
Ultrasonic timeout	12 ms (2 m range)	25 ms	Kept the library default; changing it is a planned optimisation (§6)
Glove IMU	MPU6050, ±250 °/s, 20 Hz filter	MPU6500, ±1000 °/s, 94 Hz filter, learned hand frame stored in flash	Fast flicks clipped; the sensor mounts in an arbitrary orientation
Link payload	Label string on BLE notify	`seq`, `pan`, `tilt`, `evSeq`, `evCode` read every 50 ms	Angles are needed continuously for drive; events piggyback on the same packet

5 Firmware structure

5.1 Layers and modules

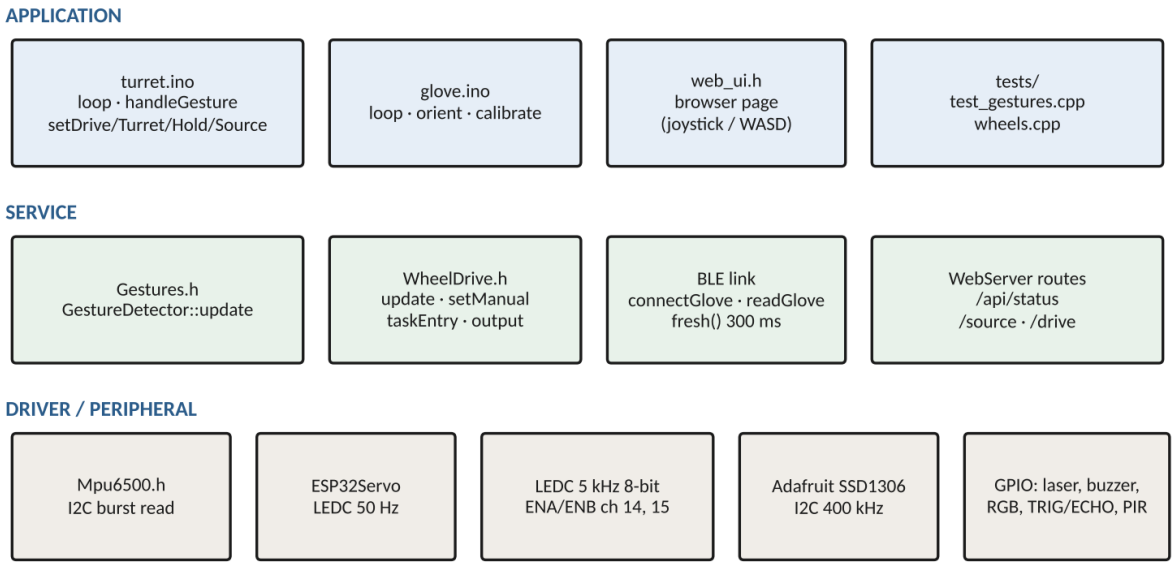


Figure 7 — Firmware module diagram.

Peripheral ownership: `Mpu6500.h` owns the glove I2C bus; the turret's `loop()` owns the OLED bus, GPIO outputs and servos; `WheelDrive` owns LEDC channels 14 and 15 and the six L298N pins, and nothing else may write them. The wheel channels are attached on a different LEDC timer from the servo channels so a 5 kHz PWM cannot disturb the 50 Hz servo signals.

5.2 Implementation of the models

FSM. Mode is held in flags and changed only through `setDrive()`, `setTurret()`, `setHold()` and `setSource()`, each of which also clears the laser and resets the wheel state. Gesture events go through a `switch` in `handleGesture()`; serial commands through a `switch` in `commands()`. **Schedule.** The loop runs jobs when `now - lastX >= T` (read 50 ms, sense 150 ms, servo 20 ms, display 250 ms, BLE rescan 2 s) and ends with `delay(2)`. The wheel output runs in a FreeRTOS task (`wheel-output`, 3072-byte stack, priority 2, `vTaskDelay(10 ms)`); state shared with it is guarded by a `portMUX` critical section. The glove loop is a plain `delay(10)` loop. **Where faults are caught:** a packet must parse as `seq,pan,tilt[,evSeq,evCode]` with pan 30–150, tilt 45–135 and event 0–6, or it is dropped; `fresh()` rejects data older than 300 ms and the loop forces the laser off each pass when it is false; the wheel task disarms if `update()` has not been called for 250 ms and ignores web commands older than 400 ms; I2C has a 25 ms timeout; a failed BLE connect disconnects and retries; the glove retries sensor start-up every 3 s and withholds BLE until the sensor answers.

5.3 Model-to-code traceability

Model element	File	Function	Commit / line
CDFG n1 IMU read	glove/Mpu6500.h	`Mpu6500::getEvent`	L23–40
CDFG n2 hand frame	glove/glove.ino	`readHand`	L54–73
CDFG n3–n5 filters, yaw	glove/Gestures.h	`GestureDetector::update`	L34–51
CDFG n6–n7 double flick	glove/Gestures.h	`GestureDetector::update`	L53–66
CDFG n8 punch	glove/Gestures.h	`GestureDetector::update`	L68–78
CDFG n9–n10 shake, sweep	glove/Gestures.h	`GestureDetector::update`	L80–100
CDFG n11 still; n12 cooldown	glove/Gestures.h	`GestureDetector::update`	L102–112
CDFG n13 packet	glove/glove.ino	`loop`	L287–312
Learned hand frame	glove/glove.ino	`orient`, `calibrate`	L149–201
FSM NO_LINK / link	turret/turret.ino	`fresh`, `connectGlove`, `readGlove`	L61–113
FSM TURRET / HOLD / DRIVE	turret/turret.ino	`setDrive`, `setTurret`, `setHold`	L115–117
FSM WEB	turret/turret.ino	`setSource`, `handleSource`, `handleDrive`	L120–155
FSM gesture actions	turret/turret.ino	`handleGesture`	L157–176
FSM DIAG	turret/turret.ino	`diagnosticTick`, `commands` ('t')	L205–275
Task: sense 150 ms	turret/turret.ino	`sense`; loop gate	L178–186; L317
Task: servo 20 ms	turret/turret.ino	`loop`	L322–330
Task: OLED 250 ms	turret/turret.ino	`loop`	L338–355
Task: wheel output 10 ms	turret/WheelDrive.h	`taskEntry`, `output`	L126–158
Arming and 250 ms watchdog	turret/WheelDrive.h	`update`, `taskEntry`	L35–58; L144–146
Web 400 ms expiry	turret/WheelDrive.h	`setManual`, `taskEntry`	L67–76; L147

The folder is not a git repository, so there are no commit hashes; the last column gives line ranges in the files as submitted (`glove.ino` *317 lines,* `Gestures.h` *139,* `Mpu6500.h` *53,* `turret.ino` *357,* `WheelDrive.h` *159,* `web_ui.h` *13,061 bytes).*

6 Optimisation

We made two deliberate latency changes on the turret and report them against what the code constants and build output show. Neither was timed on a scope, so the latency rows are design values. Flash and RAM come from the compiler's own output; they went **up**, and we accept that.

Metric	Baseline	Optimised	Change	Mechanism	Requirement
Flash (bytes)	1,150,902 (wheel build, 26 Oct)	1,749,338 (web build, Huge APP, 27 Oct)	+598,436 (+52.0%)	WiFi access point and WebServer libraries added; not an optimisation	R-F4
RAM (bytes)	43,492	65,068	+21,576 (+49.6%)	Same; 19% of 327,680 B used	R-F4
Critical-path latency: loop stall in web mode	≥ 1000 ms every 2 s (blocking BLE scan `scan->start(1)`)	≤ 85 ms (model of one loop pass)	−915 ms or more	Glove scan and reads are skipped when the source is WEB	R-F4, R-N1
Critical-path latency: tilt to wheel motion (arming)	500 ms neutral hold, then duty 61–64 (below motor start)	first neutral sample arms; duty starts at 90 (`MIN_PWM`) and ramps to 200	−500 ms before first motion	Removed the hold timer; skip the motor dead zone	R-F3, R-N1
Average current	not measured	not measured	—	—	R-N4

Trade-off accepted: the web build uses half as much flash again and requires the Huge APP partition; the hold-free arming can let the car move if the glove reconnects with the hand already near flat. The duty change

and the arming change are in the source; the arming change and the newest glove firmware were not yet flashed when this report was written, and the floor drive test with the new duty was not completed.

7 Test plan and measured results

Where a row says *host* the test is a C++ program built with g++ on the firmware headers, with a fixed random seed; it was re-run for this report, so repeats are identical by construction and the spread is the variation built into the traces. Rows marked *not run* are listed so that no requirement is left without a test.

Test ID	Requirement	Method and equipment	Expected	Measured (n, mean, spread)	Pass / fail
T1	R-F1	Glove flat and still after calibration; angles logged over serial at 115200 (`glove-test.log`, 56 lines, ≈28 s), PC + USB	Mean within ±3° of 90°, sd ≤ 2°	n = 56. Pan 91.0°, sd 1.3° (89–95). Tilt 89.3°, sd 0.9° (87–91)	Pass (rest only)
T1b	R-F1	Hand tilted, serial output observed	Roll right → pan > 90; fingers down → tilt < 90	Both observed, n = 1 each	Pass
T2	R-F2	*Host*: 300 randomised synthetic trials per gesture (`test_gestures.cpp`, amplitude, timing, noise vary)	≥ 90% exact match	n = 300 each: double flick 100.0%, punch 100.0%, shake 100.0%, sweep L 94.7%, sweep R 95.3%	Pass (synthetic)
T2b	R-F2	*Host*: 13 deterministic cases	All pass	12 of 13. FAIL: "sweep (yaw +) → SWEEP_LEFT"	**Fail**
T3	R-F2	*Host*: one simulated hour of tilting, steering, walking, holds	0 false actions	0 false action gestures, 0 STILL events	Pass (synthetic)
T3b	R-F2	Real glove: gestures performed, glove and turret serial lines read	Each gesture detected	Double flick, shake, punch detected and acted on; sweeps not detected. Counts were not logged	**Partial**
T4	R-F3	*Host*: `wheels.cpp` with a fake clock: startup off, tilted hand cannot arm, first neutral arms, 150 ms reversal dead time, speed cap, link loss, stop	All assertions hold	"PASS: startup … hold-free arming", n = 1 deterministic run	Pass
T4b	R-F3	Wheels lifted; pin read-back (`p`) during 1.5 s pulses at duty 180: left/right, forward/reverse	Correct IN/EN levels; all zero after expiry	4 of 4 pulses correct; all pins 0 afterwards. User reported wheels turned ("all good"); not measured. Glove-driven floor test not completed	Pass (pins); floor *not run*
T5	R-F4	*Host*: web cases in `wheels.cpp` (direction, dead time, 400 ms timeout, steer cap, stop); page JavaScript parsed with `node --check`	Page works on a phone; stop ≤ 400 ms	Host cases pass; JS parses. Phone test not reported	**Partial**
T6	R-F5	HC-SR04 range printed over serial (`wheel-test-extended.log`), no fixed target	Repeatable to ±1 cm at a fixed target	n = 46 lines, 45 valid: mean 37.4 cm, sd 15.8, range 19.9–83.2; one no-echo. PIR not tested	**Fail** (erratic)
T7	R-N1	Planned: GPIO toggle on glove at decision and on turret at actuation, logic analyser	≤ 300 ms	Not measured. Estimate 0.2–0.3 s (filter τ ≈ 160 ms, 40 ms packets, 50 ms poll)	*Not run*
T8 (fault)	R-N2	(a) Turret booted with the glove absent; (b) *host*: no `update()` for 251 ms; (c) *host*: `update(live=false)` while moving	Wheels off, laser off, ≤ 250 ms	(a) `link=0 armed=0`, laser off, wheels still, many lines in the device log. (b), (c) outputs 0/0 and disarmed. Stale-link laser path not run on hardware	Pass (host and bench)

Test ID	Requirement	Method and equipment	Expected	Measured (n, mean, spread)	Pass / fail
T9	R-N3	Sum of the three receipts of 28-09-2026	≤ ₹5,000	n = 3: ₹110 + ₹2,703 + ₹649 = ₹3,462	Pass
T10	R-N4	Untethered run time	≥ 10 min	Both boards on USB; no battery path built	**Not met**

7.1 Timing measurements

Task	WCET (Review 2)	WCET measured	Jitter	Deadline met?
IMU read + gesture update (glove, 10 ms)	≈ 24 ms for a window classifier; 1.6 ms for the read	not measured (estimate 1.67 ms)	not measured; 13 packets per ≥ 500 ms status interval in `glove-test.log`, consistent with a 40 ms period	Unknown; model says yes (slack 8.3 ms)
BLE read (turret, 50 ms)	2 ms (notify)	not measured (estimate 30 ms)	not measured; model start-to-start 52–85 ms	Unknown; model says late under coincidence
Servo step (20 ms)	0.05 ms	not measured	not measured; model start-to-start up to 100 ms	Model says no
HC-SR04 sense (150 ms)	12 ms	not measured (bound 25.012 ms from the code constant)	not measured	Unknown; model says yes
OLED refresh (250 ms)	28 ms	not measured	not measured	Unknown; model says yes
Wheel task (10 ms) and 250 ms watchdog	0.1 ms	pulse expiry only: 1.5 s pulse seen high at 0.94 s and low by 2.36 s after a 0.34 s start (0.5–1.4 s sampling)	—	Yes at that resolution; not a scope measurement

No logic-analyser or scope capture exists yet, so no capture is shown here. The planned capture is a GPIO toggle at the start and end of each job in the table above; until it is taken, every "model says" entry is a prediction, not a result.

7.2 Discussion

The logic that guards the car and the laser is tested best: the arming, watchdog, dead-time and web-timeout rules pass their host tests, and the four driver channels were read back correctly on hardware. The gesture detector does what the requirement asks on synthetic traces, but those traces are our own model of a wrist; on the real hand the double flick, shake and punch worked and the sweeps did not, and one deterministic sweep case still fails in the newest source. So R-F2 is **not** demonstrated on a real wrist. The user's own sweep is an edge-on swat with 10–14 rad/s rotation about the vertical, which is outside the thresholds tuned on the synthetic data (4.5 rad/s). The ultrasonic readings spread from 19.9 to 83.2 cm with no fixed target, so R-F5 is not met and we cannot say whether the cause is the wiring, the divider, the 5 V rail or the target. R-N1, R-N4 and the timing rows are simply unmeasured. The timing model suggests the turret loop is the weak part of the design: a 25 ms ultrasonic timeout and a 28 ms display transfer inside a non-preemptive loop slow the servo ramp, which is consistent with the sluggish pan-tilt we expect but have not measured.

8 Design-decision log

ID	Date	Decision	Alternatives rejected	Reason and evidence	Consequence
D1	Sep 2026	ESP32 for both boards	Arduino Nano + HC-05; Raspberry Pi Pico W	§2.1 matrix: only option with BLE, WiFi, 17+ pins and FPU on one board	WiFi AP, BLE and 5 V draw all on one chip; USB-only power for now

ID	Date	Decision	Alternatives rejected	Reason and evidence	Consequence
D2	Sep 2026	BLE GATT read for the glove link	ESP-NOW; WiFi UDP	Proposal link; verified 26 Oct (glove `BLE=CONNECTED`, turret `link=1`). Matrix margin to ESP-NOW is 0.1	Polling adds up to 50 ms; 1 s synchronous scan stalls the loop when the glove is off
D3	17 Sep 2026	Laser must be a KY-008 module with its resistor	Three bare 5 V diode listings	Bare diodes have no resistor and risk the diode or the GPIO (BOM verification)	Laser pin switches a module input only
D4	28 Sep 2026	Buy MG995, SG90, RGB modules, 4WD chassis kit as on the receipts	MG996R, WS2812 ring (spec)	What was available at the shop; receipts total ₹3,462	RGB is one 3-pin module (GPIO32/33/4), no ring; bracket/SG90 mismatch still open
D5	26 Oct 2026	Separate MPU6500 init path in `Mpu6500.h`	Replace the sensor	I2C scan: device at 0x68 with WHO_AM_I 0x70 (MPU6500), the MPU6050 library rejected it	Both chips supported; same wiring
D6	26 Oct 2026	L298N with ENA/ENB on GPIO2/15 (jumpers removed); wheel output in its own task with 250 ms watchdog	Jumper-capped full speed; updating wheels from `loop()` only	"Car runs full speed on battery" fault; host watchdog test; the loop can block for 1 s during a BLE scan	GPIO2 can block flashing; extra RTOS task and mutex
D7	27 Oct 2026	Rule-based gesture detector	TinyML (Edge Impulse), 4 baseline + 4 stretch classes; DTW	No dataset collected; 100% on 3 of 5 gestures synthetic; real hand partial (T2, T3b)	AI component of the proposal not delivered; sweeps need re-tuning
D8	27 Oct 2026	Web control via WiFi SoftAP and HTTP polling	Join a router; WebSocket	Works with no network in the field; no extra library; 400 ms failsafe in the wheel task	+598 KB flash, Huge APP partition; default AP password must be changed
D9	27 Oct 2026	First neutral sample arms the wheels	Keep 500 ms neutral hold	User asked to remove the wait; host test updated	Car may move on glove reconnect near flat; not yet flashed
D10	27 Oct 2026	STILL gesture reported only	Keep STILL stopping the car	It conflicted with the level hold used to arm the wheels	Return to level is the only stop gesture

9 Bill of materials and cost

Item	Part number	Qty	Unit cost (₹)	Source	Total (₹)
ESP32 dev board, 30-pin (CP2102)	Robocraze NodeMCU ESP32	2	450	Amazon.in / shop	900
Tower Pro MG995 servo (pan)	MG995	1	350	Receipt 28-09	350
SG90 micro servo (tilt)	SG90	1	130	Receipt 28-09	130
IMU breakout (chip is MPU6500)	GY-521 / MPU6050	1	150	Receipt 28-09	150
HC-SR04 ultrasonic	HC-SR04	1	130	Receipt 28-09	130
HC-SR501 PIR	HC-SR501	1	120	Receipt 28-09	120
Laser module	KY-008 type	1	90	Receipt 28-09	90
Buzzer, 5 V active	—	1	40	Receipt 28-09	40
0.96" OLED, I2C	SSD1306 128×64	1	250	Receipt 28-09	250
4WD acrylic chassis kit with 4 TT motors	4WD kit	1	649	Receipt 3	649
RGB LED module	—	2	n/i	Receipt 28-09	n/i
18650 cells (2500 mAh), 4-cell holder, 2-cell charger	ICR-18650-2500	4 + 1 + 1	n/i	Receipt 28-09	n/i

Item	Part number	Qty	Unit cost (₹)	Source	Total (₹)
Breadboards 400 and 840 pt, 6×4" perfboard, Berg strips, jumpers	—	see receipts	n/i	Receipt 28-09	n/i
L298N dual motor driver	L298N	1	n/i	added for wheels	n/i
Total	**₹3,462** paid (receipts ₹110 + ₹2,703 + ₹649). Itemised rows above ≈ ₹2,809; the remaining ≈ ₹653 covers the n/i rows.				

n/i = not itemised on the receipts. Unit costs are the specification's estimates, not receipt lines; the receipts give totals only. The ₹649 chassis kit is the third receipt. Still to buy before an untethered build: a buck or boost converter matched to the measured pack voltage, a LiPo cell with charger for the glove, and the 1 kΩ/2 kΩ divider resistors.

10 Individual contribution

Member	Contribution	Evidence (files, commits)	Report sections	Share %
Sankepalli Kharanshu	Complete hardware; calibration of the main (turret) ESP32	Chassis, wiring and breadboard build (Figure 3); `docs/pan_tilt_DIY_diagram.png`, `images/chassis-photo.png`; turret pin map in `turret/turret.ino` and `docs/Full_Wiring_Connections.md`; centre and range commands in the turret serial menu	§3.2, §3.4, §9	25
Tom Biju	Glove code calibration	`glove/glove.ino` `orient` and `calibrate` (L149–201), flash-stored hand frame; resting-angle log `glove-test.log` (T1)	§4.1, §7 T1–T3	25
Ahmed Sahil	Documentation and report creation	This report and the Review 2 document; `docs/PROGRESS_2026-10-27.md`, `DEVICE_TEST_RESULTS.md`, `GLOVE_VERIFICATION.md`	§1, §2, §8, §10, §11, abstract, assembly	25
Harish Saravanan	Wheel-drive testing	`tests/wheels.cpp` host cases (T4, T5); `wheel-test.log`, `wheel-test-extended.log`, pin read-back samples (T4b)	§6, §7 T4–T5	25

Shares are the proposal's equal 25% split; change them if the real effort differed. Each member must be able to defend their contribution at the presentation, using the evidence files and the line ranges in §5.3.

11 Limitations and future work

Review 1 (reviewer: Nimal J Kumar) approved the proposal with conditions. Status against each condition and mandatory addition:

Review 1 item	Status	Evidence or gap
Baseline of four gestures; others optional	**Met on synthetic data, not on the real wrist**	Five action gestures implemented (double flick, shake, punch, sweep left/right). Only double flick, shake and punch worked on the real hand, so the demonstrated baseline is three (§7 T3b)
Drop the "sentry" and projectile framing	Met in this report	Presented as a tracking and pointing platform (§1.1). The approved title, folder name and WiFi password string still contain "sentry"
Supply Tom Biju's email	Completed	Supplied by the team on 27 Oct 2026
R1: written BLE packet specification	Met	Table in §3.1
R1: servo control and glove IMU capture working independently	Partly	IMU capture: logged (`glove-test.log`, T1). Turret servo test exists as the `t` hardware test; a pass result was not recorded
R1: power budget and a separate servo supply rail on the wiring diagram	Partly	Budget in §3.3 (estimated, not measured). Servo V+ from an external 5 V rail with common ground is on the schematic and pin map; the rail was not built or measured
R2: rule-based gesture fallback	Met	It is now the primary method (§2.3, `Gestures.h`)

Review 1 item	Status	Evidence or gap
R2: confusion matrix on held-out gesture data	**Not met**	Only per-gesture accuracy on synthetic traces (T2). No held-out real data was collected
R2: measured glove-to-turret latency and BLE range	**Not met**	Neither measured (T7); estimate only
R2: safe state: on BLE loss the turret centres, stops and says so on the OLED	**Partly**	Laser off, wheels stop, OLED shows `GLOVE LINK LOST`. The servos hold their last position instead of centring

What did not work or fell short. (1) Sweeps are not detected on the real hand and one deterministic sweep test fails; the real-wrist accuracy of every gesture is unmeasured (R-F2). (2) The TinyML component of the proposal was not built. (3) AUTO tracking from the ultrasonic and PIR sensors was not implemented; they only raise alerts. (4) The ultrasonic readings are erratic (R-F5). (5) Latency, WCET, jitter and current were never measured, so R-N1, R-N4 and §7.1 rest on estimates. (6) No battery path exists: both boards run on USB. (7) The pan-tilt mount, laser and glove strap are not mounted; the laser was never fired and the PIR never tested. (8) The web page has not been run on a phone. (9) The BLE link is unauthenticated, the WiFi password is the default, and `wheels.cpp` crashes at `-O1` for a reason we have not diagnosed. (10) A synchronous one-second BLE scan stalls the loop whenever the glove is off in glove mode.

What we would do differently. Collect gesture data on day one and keep the rules only as the fallback; measure the pack voltage before buying a regulator; capture timing from the first week instead of the last; mount the mechanics before writing the drive logic.

Next engineering step. Take the GPIO-toggle captures for §7.1; set the ultrasonic timeout to 12 ms and the I2C clock to 400 kHz (the model says this cuts the glove read from 1.6 ms to about 0.4 ms); try ESP-NOW against BLE for latency; re-tune the sweep thresholds on the user's own edge-on swat and rerun T2b; then add a measured buck converter and run the 10-minute untethered test.

References

[1] Espressif Systems, *ESP32 Series Datasheet*, espressif.com.
[2] InvenSense, *MPU-6500 Product Specification*, register map WHO_AM_I = 0x70.
[3] STMicroelectronics, *L298 Dual Full-Bridge Driver Datasheet*, st.com/en/motor-drivers/l298.html.
[4] Espressif, *Arduino-ESP32 BLE API*, docs.espressif.com/projects/arduino-esp32/en/latest/api/ble.html.
[5] Espressif, *Arduino-ESP32 LEDC API*, docs.espressif.com/projects/arduino-esp32/en/latest/api/ledc.html.
[6] ESP32Servo library, github.com/madhephaestus/ESP32Servo.
[7] Adafruit, *Adafruit_MPU6050*, adafruit.github.io/Adafruit_MPU6050; *Adafruit SSD1306 and GFX* libraries.
[8] Elec Freaks, *HC-SR04 Ultrasonic Module User Guide*; *HC-SR501 PIR Module Datasheet*.
[9] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, 1973.
[10] Edge Impulse, *Motion classification tutorial* (considered and not used), docs.edgeimpulse.com.

Appendices

A Source code

Repository: *[add URL]*. Local copy: `ESA_Friend_Project/Gesture_Sentry_Firmware_and_Guide/gesture-sentry/`.

File	Lines	Role
glove/glove.ino	317	Glove loop, orientation learning, calibration, BLE peripheral
glove/Gestures.h	139	Rule-based gesture detector
glove/Mpu6500.h	53	MPU6500 driver
turret/turret.ino	357	Turret loop, BLE central, mode logic, WiFi AP and web routes, diagnostics
turret/WheelDrive.h	159	L298N driver, arming, watchdog, web manual drive
turret/web_ui.h	13,061 B	Browser control page
tests/test_gestures.cpp, tests/wheels.cpp	202, 82	Host tests

Flashed firmware SHA-256 values recorded in the project: turret diagnostic build `eb9d34e0…a07d1`, *glove build* `63366b90…f1adf37`, *wheel build* `deba1d53…e5545f5`.

B Raw data

File	What it holds
glove-test.log	Glove status lines: seq, roll, pitch, pan, tilt, BLE state (T1)
wheel-test.log, wheel-test-extended.log, wheel-test-upload.log	Wheel pulse snapshots and ultrasonic readings (T4b, T6)
docs/session-2026-10-27-web-control/logs/wheel_pin_samples_2026-10-27.txt	Pin read-back during the four 1.5 s pulses (T4b)
DEVICE_TEST_RESULTS.md, GLOVE_VERIFICATION.md, VERIFICATION.md	Per-device results and limitations
docs/PROGRESS_2026-10-27.md	Faults found, glove orientation, gesture results, open bugs
docs/SESSION_2026-10-27 (web-control)/SESSION_RECORD.md	Web control design, flash attempts, decisions

C Review 2 models (changed substantially)

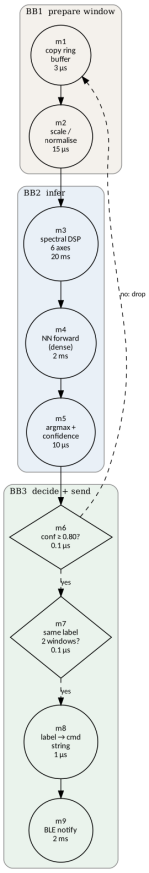


Figure C1 — Review 2 CDFG: window classifier (estimated costs).

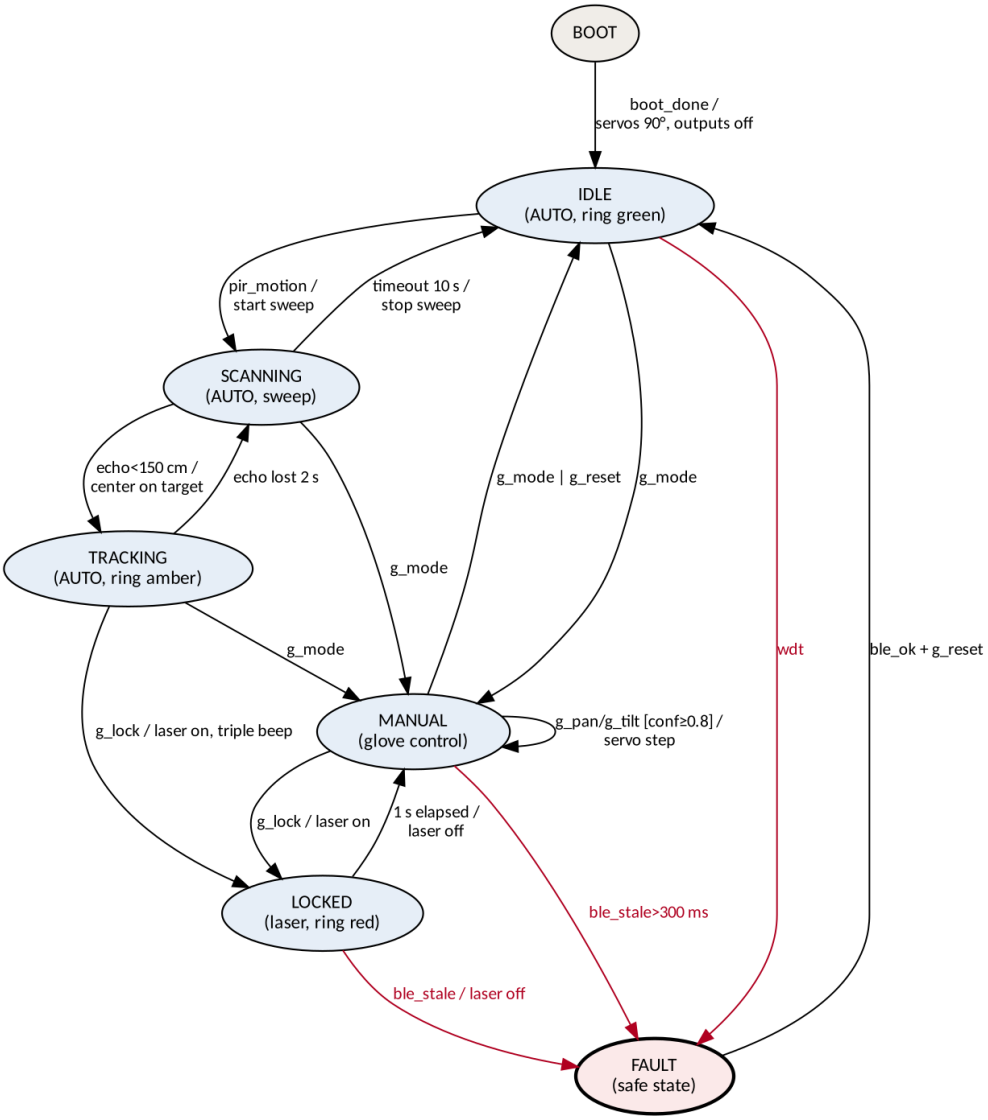


Figure C2 — Review 2 system FSM (AUTO / MANUAL / LOCKED / FAULT).

Submission checklist

Item	✓
15–20 pages excluding front matter and appendices; guidance blocks deleted	✓
Every requirement has an ID, a measurable target and a test	✓
At least two alternatives compared for each major decision	✓
Models revised to match the build; changes since Review 2 listed	✓
Traceability table from model elements to code	✓
Optimisation with measured before and after	Partly: flash/RAM from build output; latency rows are design values
Repeated measurements with spread; one fault-injection test	Partly: T8 is host + bench, n = 1; scope data missing
Decision log, BOM and contribution table complete	✓ (BOM unit costs for some rows not itemised)